

Lightsaber_X: simulation-layer modifications with respect to the reference simulator

Technical note — environment, reward, and controller handover

M. Grillo

July 9, 2026

Abstract

This note documents, at implementation level, how the `Lightsaber_X` code base differs from the trusted reference simulator (`old_ref`) in the three areas that constitute the online controller-selection study: (i) the construction of a *non-stationary* noise environment out of three calibrated stationary regimes; (ii) the definition and streaming computation of a scalar *reward* that scores closed-loop performance on consecutive time windows; and (iii) the *bumpless handover* machinery that lets the active angular controller be exchanged at run time without corrupting the loop state. The decision layer that chooses which controller to play (the bandit) is deliberately *not* described here; throughout, it is treated as a black box that returns an arm index $a_t \in \{0, 1, 2\}$ once per decision window. All physics of the underlying plant is unchanged with respect to the reference; the equivalence is established in `VALIDATION_vs_old_ref.md` and summarised in Section 1.

Contents

1	Baseline and scope	2
2	Non-stationary noise environment	2
2.1	The three stationary regimes	2
2.2	Regime mixture: the weight process $W(t)$	3
2.3	Months-scale realisation: the disk-backed noise cache	4
3	Reward generation	4
3.1	Multi-band features	4
3.2	Streaming (stateful) filtering	5
3.3	From features to a scalar score	5
3.4	Normalisation to $[0, 1]$ and calibration	6
4	Online controller switching	6
4.1	The controller variants	6
4.2	Why switching is delicate	6
4.3	Production mechanism: parallel banks with a bumpless ramp	7
4.4	State continuity across the whole horizon	7
5	The fast engine	8
6	Summary of differences vs. the reference	8

1 Baseline and scope

`old_ref` is the original component-graph simulator (`System/Mirror/Beam/...` classes) of the Sidles–Sigg angular-control problem for a Fabry–Perot arm cavity. `Lightsaber_X` is built on its author’s Plant-class refactor (`Lightsaber.py: Plant, Sensors, SS_compensation, Controller`), which we validated against `old_ref` element by element. The core plant physics is *identical*:

- the actuation path (PUM pitch $\rightarrow$ test-mass pitch) and the radiation-pressure path (test-mass pitch $\rightarrow$ pitch) use the same zero–pole–gain models ($k = 93.52955$ and $k = 2.567652$ respectively, identical zeros/poles);
- the suspension transfer functions are bit-identical tabulated data;
- the intra-cavity power follows the same Fabry–Perot response $P_{\text{cav}} = P_{\text{in}} t_{\text{ITM}} / |1 - \rho_{\text{ITM}} e^{4\pi i \delta L / \lambda}|^2$, with the cavity length fluctuation δL high-passed by an elliptic filter `ellip(2, 1, 140, 2 $\pi$  · 50, 'high')`;
- the optical parameters are unchanged: $P = 705$ W, $L = 3994.5$ m, $R_{\text{ITM}} = 1934$ m, $R_{\text{ETM}} = 2245$ m, $t_{\text{ITM}} = 0.014$, $\lambda = 1064$ nm, $f_s = 256$ Hz.

Everything documented below sits *on top* of this validated plant: the reference simulator runs one stationary noise batch with one fixed controller; `Lightsaber_X` runs a continuous, months-long, regime-switching environment in which the hard-mode controller may be exchanged at any decision boundary. A Numba just-in-time port of the per-sample loop (`fast_engine.py`, Section 5) makes such horizons computationally feasible; it reproduces the Plant-class per-sample sequence exactly and is validated against it.

Two soft/hard conventions used throughout: the two mirrors’ local pitch coordinates ($\theta_{\text{ITM}}, \theta_{\text{ETM}}$) are mapped to the *soft/hard eigenbasis* by

$$M_{\text{loc} \rightarrow \text{eig}} = \frac{1}{1+r^2} \begin{pmatrix} 1 & r \\ -r & 1 \end{pmatrix}, \quad r = \frac{1}{2} \left(g_1 - g_2 + \sqrt{(g_1 - g_2)^2 + 4} \right), \quad g_i = 1 - L/R_i, \quad (1)$$

and the *hard* mode — the one destabilised by radiation pressure — is the channel that the exchangeable controller acts on and the reward scores. The soft-mode controller is common to all variants and is never switched.

2 Non-stationary noise environment

2.1 The three stationary regimes

The reference simulator draws one stationary noise realisation from fixed O3-derived spectra. `Lightsaber_X` replaces this with a *family of three* calibrated environmental states (“regimes”), each defined by (a) a seismic input amplitude spectral density (ASD) file and (b) a set of scalar multipliers applied to the default plant flags (`bandit_simulation.py`):

The per-regime noise itself is synthesised exactly as the reference plant does it (`Plant.create_tst_noise_from_top`): in the frequency domain, complex white Gaussian amplitudes are shaped by (root-PSD $\times$ suspension transfer function) and transformed back,

$$n^{(j)}(t) = f_s \cdot \text{irfft} \left[\eta(f) s_j \sqrt{S_j(f)} T_j(f) \right], \quad \eta(f) = \eta_{\text{re}} + i \eta_{\text{im}}, \quad \eta \sim \mathcal{N}(0, \tfrac{1}{2} \sqrt{T_{\text{blk}}}), \quad (2)$$

per noise component j (ITM/ETM seismic through the top-stage TFs, OSEM length/pitch through their TFs), where s_j is the regime scale of Table 1 and T_{blk} the synthesis block length.

Regime	ASD file	seismic	input power	sensing
R_0 (nominal)	ASD_R0_nominal.csv	1.0	1.00	120.0
R_1 (high microseism)	ASD_R1_high_micro.csv	6.0	0.95	12.0
R_2 (extreme seismic)	ASD_R2_high_micro2.csv	20.0	0.90	0.25

Table 1: Regime definitions. The seismic scale multiplies the ITM/ETM ISI injection; the sensing scale multiplies the white readout-noise ASDs ($n_{\text{hard}} = 3 \times 10^{-14}$, $n_{\text{soft}} = 1 \times 10^{-13}$ in eigenbasis units/ $\sqrt{\text{Hz}}$). R_2 pairs *strong* seismic input with a very *clean* sensor, R_0 the opposite — so no single fixed controller is best in all regimes, which is the premise of the study.

The six shaped components are summed into the two injected test-mass streams (ITM, ETM). The laser intensity noise is produced the same way from the O3 relative-power PSD and enters as $P_{\text{in}}(t) = P(1 + f_s \cdot \text{irfft}[\eta \sqrt{S_{\text{RIN}}}]$. The DC bin is zeroed. This reproduces the reference construction *block by block*; each block is an independent stationary realisation of the regime PSDs.

2.2 Regime mixture: the weight process $W(t)$

Non-stationarity is introduced by blending the three regime streams with a time-dependent, simplex-valued weight vector $W(t) = (W_0(t), W_1(t), W_2(t))$, $\sum_i W_i = 1$:

$$n(t) = \sum_i W_i(t) n_i(t), \quad P_{\text{in}}(t) = \sum_i W_i(t) P_{\text{in},i}(t), \quad s_{\text{sens}}(t) = \sum_i W_i(t) s_{\text{sens},i}, \quad (3)$$

i.e. the injected suspension noise, the input power, *and* the readout-noise scale all follow the same mixture. Crucially, the three per-regime realisations $n_i(t)$ are generated *coherently*: within each synthesis block the random generator is reset to the same seed before shaping each regime, and the draw order (power first, then the six test-mass components) is fixed, so the regimes differ *only* through their spectral shaping and scales, not through independent randomness. A weight crossing $W : (1, 0, 0) \rightarrow (0, 0, 1)$ therefore morphs the statistics of one underlying realisation rather than cross-fading two unrelated noise records (which would transiently *reduce* the variance by up to a factor 2 at the midpoint).

The weight process itself (`RegimeEnv` in `bandit_experiment.py`) is a compressed-timescale model of a day at a detector site, with two independent layers:

(a) **Slow diurnal drift** ($R_0 \leftrightarrow R_2$). A “peaky” diurnal profile

$$v_*(t) = \frac{1}{2} + \frac{1}{2} \text{sign}(\cos(2\pi t/P_d)) |\cos(2\pi t/P_d)|^{0.33} \quad (4)$$

(period P_d ; the exponent 0.33 makes the profile dwell near its extremes and cross quickly) is *not* followed continuously. Instead, per diurnal cycle a Poisson number ($\mu = 20$) of *transition events* is drawn, with start times normally scattered ($\sigma = P_d/8$) around the two steep points of the cycle ($P_d/4$ and $3P_d/4$). At each event the drift value ramps *linearly* from its current value to the target $v_*(t_e)$ over an exponentially distributed ramp time (mean 40 s); between events it holds constant. The result is a piecewise-linear, stochastically timed day/night oscillation. A hard threshold then snaps near-extreme values to the pure regimes ($v > 0.75 \mapsto 1$, $v < 0.25 \mapsto 0$), so the environment spends most of its time in a *pure* R_0 or R_2 state connected by genuine finite-duration ramps — transitions are continuous, not instantaneous.

(b) **Microseismic spike events** (R_1). Independently, R_1 “storms” arrive as a Poisson process (rate: `spike_rate` events per diurnal period; production 6-month run: 1 per day). Each event is a trapezoid in the R_1 weight: attack, hold, and decay durations drawn exponentially with configurable means (defaults 30 s/150 s/30 s; production 6-month run uses mean hold = 1500 s).

Composition. With $v(t)$ the thresholded drift value and $w_1(t) \in [0, 1]$ the spike envelope, the final weights are

$$W(t) = \left(v(t) (1 - w_1(t)), w_1(t), (1 - v(t)) (1 - w_1(t)) \right), \quad (5)$$

i.e. a spike overrides the diurnal state and then hands back to it. The process is seeded, hence fully reproducible; all its parameters (`spike_rate`, `spike_attack/hold/decay`, μ , ramp mean, jitter, exponent, threshold) are part of the environment configuration.

2.3 Months-scale realisation: the disk-backed noise cache

At $f_s = 256$ Hz a 6-month horizon is $N \approx 4 \times 10^9$ samples; three regime streams in RAM would exceed 100 GB. Because the environment is *policy-independent* — every candidate policy must see the identical noise (common random numbers) — the blended streams of Eq. (3) are computed *once* and stored (`bandit_noise_cache.py`): the two blended test-mass streams and the blended input power are written as float32 memory-mapped arrays (`sus_itm.npy`, `sus_etm.npy`, `power.npy`, ~ 48 GB for six months), together with a manifest holding the per-second weight schedule $W(t)$, the sensing-scale schedule, the seed, and all environment parameters (which form the cache key: changing any parameter forces regeneration). Synthesis proceeds in 4096 s blocks; per block, per regime, one call to Eq. (2) with the block seed `seed + 1000 b`, then the sample-accurate blend with W interpolated from the per-second schedule. The only artefact of block-wise synthesis is a short filter transient at each block boundary, negligible on a months-long horizon (the slowest reward band settles in ~ 60 s, cf. Section 3).

The white *readout* noise is deliberately *not* stored: it is regenerated at run time, per decision window, from a fixed seed shared by all policies, as Gaussian samples of standard deviation

$$\sigma_{\bullet}(t) = n_{\bullet} \sqrt{f_s/2} \cdot s_{\text{sens}}(t), \quad \bullet \in \{\text{hard}, \text{soft}\}, \quad (6)$$

added to the two eigen-mode readouts. Since every policy consumes the same number of draws per window in the same order, the sensing-noise realisation is *identical across policies* — the closed-loop trajectories differ only through the controller choices.

3 Reward generation

The reference simulator has no notion of reward. **Lightsaber_X** scores closed-loop performance on consecutive windows of T_{hold} seconds (production: 100 s); the score is designed to (i) reflect both residual motion *and* control effort across the band, (ii) be computable *online* from streaming data without acausal processing, and (iii) be comparable across regimes.

3.1 Multi-band features

Two signals are scored, both in the eigenbasis of Eq. (1): the hard-mode error signal $e_h(t)$ (the test-mass pitch projected on the hard mode — the controller’s input before readout noise) and the hard-mode control effort $u_h(t)$ (the eigen-projection of the commanded actuation). Each is split into eight frequency bands by fourth-order Butterworth filters (one low-pass, seven band-passes):

band	edges [Hz]
LP	0 – 0.3
BP ₁	0.05 – 0.3
BP ₂	0.3 – 1
BP ₃	1 – 3
BP ₄	3 – 10
BP ₅	10 – 30
BP ₆	30 – 80
BP ₇	80 – 100

giving 16 features per time block: the RMS of each filtered signal over consecutive 2-second blocks (512 samples), $F_i(b) = \text{RMS} [x_i^{\text{filt}}]_{\text{block } b}$.

3.2 Streaming (stateful) filtering

A defect of an early implementation was that the band filters were re-initialised for every scoring window, so each window began with a spurious filter start-up transient — particularly harmful for the sub-0.3 Hz bands whose settling time is $\mathcal{O}(60\text{ s})$, comparable to the window itself. The current implementation (`reward_stream.StreamReward`) keeps the internal state z_i of *every* band filter *across window boundaries*: one `StreamReward` object lives for the whole closed-loop run and its `score()` method continues each `sosfilt` from the previous window’s final conditions. The reward signal is therefore filtered as one continuous stream over the entire (months-long) horizon; window boundaries have no effect whatsoever on the filtering. Equivalently, low-frequency reward content is attributed to the window in which it occurs, with no per-window warm-up bias.

3.3 From features to a scalar score

The raw score of block b is a negatively weighted log-feature sum,

$$Z(b) = \sum_{i=1}^{16} w_i \log(F_i(b) + \varepsilon), \quad R_{\text{raw}}(b) = -Z(b), \quad \varepsilon = 10^{-30}, \quad (7)$$

so that *smaller* band-RMS (less residual motion, less control effort) gives a *larger* score, and each band contributes through its *logarithm*, i.e. relative (dB-like) changes are weighted rather than absolute power — a factor-2 improvement in a quiet band counts as much as a factor-2 improvement in a loud one. The weights $w_i \in [0, 1]$ are fixed numbers stored in `bandit_reward_weights_rel2base.npz`; they were characterised offline relative to the nominal controller C_0 (hence `rel2base`) so as to emphasise the bands in which the controller variants actually differ (the file tags its reference as `base_controller = C0_nominal`). The same weight file is used by *every* script in the repository — calibration, experiment drivers, figures — so all reported numbers share one reward definition.

The *window* score is the mean of its per-2s-block raw scores; a 100 s window therefore averages 50 blocks. In the settled regimes the raw score lies in $\approx [165, 169]$ (Table of Section 5 characterisation runs), dipping to ≈ 150 during regime transitions and storms.

3.4 Normalisation to $[0, 1]$ and calibration

The decision layer consumes a bounded reward. Two maps are implemented by the function `apply_reward` in `bandit_calibrate.py`:

$$\text{logistic (default): } \rho = \left(1 + e^{-(\bar{R}_{\text{raw}} - c)/s}\right)^{-1}, \quad c = 167.5, \quad s = 2.5; \quad (8)$$

$$\text{linear min-max: } \rho = \text{clip}\left(\frac{\bar{R}_{\text{raw}} - \text{lo}}{\text{hi} - \text{lo}}, 0, 1\right). \quad (9)$$

The min-max endpoints are *not* hand-set: `bandit_calibrate.py` runs all nine steady (regime, controller) cells through the actual fast engine (1200 s each, first 60 s discarded), *plus* one fixed-controller pass over a short *drifting* environment containing regime transitions and R_1 spikes, and sets `[lo, hi]` to the pooled min/max widened by a 10% margin. The drifting pass matters: the steady cells alone bottom out around $\text{raw} \approx 163$, but real transitions dip to $\text{raw} \approx 150$; an endpoint set from steady data only would clip $\sim 10\%$ of windows to exactly 0, and a clipped-flat reward during a recovering transient is indistinguishable from a settled (very bad) regime — which was observed to corrupt change-detection logic downstream. With the widened floor a recovering transient remains a strictly increasing ramp.

The same calibration run also produces (i) the 3×3 table of mean normalised rewards per (controller, regime) — used by the Oracle reference policy and for plotting, and (ii) $\hat{\sigma}$, the median within-cell per-window reward standard deviation, which downstream algorithms use as their noise scale. By construction the table’s per-regime argmax is the designed diagonal (C_0 best in R_0 , C_1 in R_1 , C_2 in R_2).

4 Online controller switching

4.1 The controller variants

The exchangeable element is the *hard-mode* feedback filter. All three variants share the reference controller’s skeleton — the 2019-era LIGO ASC zpk (4 zeros $-0.3436 \pm 4.11j$, $-0.7854 \pm 9.392j$; 4 poles $-78.77 \pm 171.25j$, -0.062832 , -628.32 ; $k = 5797.86$), an 8th-order boost stage, an integrator zero at $-2\pi \cdot 0.1$, and two elliptic low-pass sections — and differ in DC gain, low-pass corner placement, and integrator leak:

Variant	DC gain	LP1 corner	LP2 corner	integrator leak f_{leak}
C_0 (nominal)	30.0	8 Hz	16 Hz	0.03 Hz
C_1 (high micro)	44.2	10 Hz	20 Hz	0.01 Hz
C_2 (high micro 2)	50.0	12 Hz	26 Hz	0 (ideal integrator)

Table 2: Hard-mode controller variants (`Controller.set_feedback_filter_hard`). LP1 = `ellip(2, 1, 40)`, LP2 = `ellip(4, 1, 10)` analog prototypes at the quoted corners; the leak moves the integrator pole to $-2\pi f_{\text{leak}}$. The analog cascade is discretised by the bilinear transform and factored into second-order sections (SOS).

Higher gain and higher corners buy low-frequency suppression (good under strong seismic input) at the price of more sensing noise re-injected into the loop (bad with a noisy sensor) — with the regime table (Table 1) this makes each variant the best choice in exactly one regime. The soft-mode filter is identical in all variants and its state is never touched by a switch.

4.2 Why switching is delicate

Each discretised controller is a cascade of ~ 10 second-order sections holding two delay states each. Naively “switching” by rebuilding the filter (as the reference code structure would suggest:

call `set_feedback_filter_hard` with a new name) resets those states to zero. The consequence is an actuation discontinuity — the loop, which was applying an $\mathcal{O}(\text{settled})$ command, suddenly applies the response of an empty filter — i.e. an actuator kick whose transient contaminates precisely the quantity the study measures (the post-switch reward). An early version of this code had exactly this defect, and the observed “cost of switching” was dominated by the artificial state reset rather than by the controller change itself.

Four handover implementations were therefore built and compared explicitly, in the script `switching_dynamics_fast.py` under `dev/` (results in the switching-transient study):

cold the incoming filter starts from zero state (the naive reset);

hot the outgoing filter’s state array is transplanted into the incoming filter (naive carry-over; the state vectors of different filters are not commensurate, so this is also wrong, just differently);

parallel the incoming filter has been running *continuously* in the background on the same input and is simply made active (its state is exactly what it would have been had it always been in charge);

ramp parallel, plus a linear cross-fade of the *applied actuation* from the outgoing to the incoming filter over a ramp window (bumpless transfer).

4.3 Production mechanism: parallel banks with a bumpless ramp

In the production engine *all three* controller banks are stepped at *every* sample, each maintaining its own SOS state on the common readout (`fast_engine.run_kernel`):

$$y_k^{(a)} = \text{SOS}^{(a)}(e_{h,k}; z_k^{(a)}), \quad a \in \{0, 1, 2\}, \quad (10)$$

so every bank is permanently “warm”. The applied hard-mode actuation is the active bank’s output; for the first N_{ramp} samples after a switch (production: $N_{\text{ramp}} = 2\text{ s} \cdot f_s = 512$) it is cross-faded from the previously active bank,

$$u_{h,k} = (1 - \alpha_k) y_k^{(a_{\text{prev}})} + \alpha_k y_k^{(a_{\text{new}})}, \quad \alpha_k = \frac{k + 1}{N_{\text{ramp}}}, \quad (11)$$

after which the new bank alone drives the loop. At the switch instant the applied command is (almost) exactly the old controller’s output, and it slews continuously to the new controller’s — no impulse enters the plant. The slow (Plant-class) engine implements the same semantics with two filters instead of three (`Controller.switch_hard`): the outgoing filter is kept running for the ramp duration and cross-faded out, the incoming filter starts cold but its start-up happens *inside* the fade, which begins at weight zero.

The measured switching transients under this mechanism are benign: in R_0 the post-switch reward dip is at most ≈ 0.6 raw-score units even for the largest gain jump ($C_0 \rightarrow C_2$, gain ratio 1.67), recovering within a few seconds, and the integrated loss over 40 s is $|\Delta| \lesssim 2$ raw units — versus dips an order of magnitude larger for the cold handover. This is what makes frequent online switching a viable strategy at all, and it is measured, not assumed (figures `fig1_R0_transitions_ramp` and `fig6_handover_modes` in the paper).

4.4 State continuity across the whole horizon

Switching correctness is necessary but not sufficient: the *plant* must also evolve continuously across decision windows. In the production driver a single persistent state carries, across every window boundary and hence across the entire multi-month run:

- the plant state: previous-sample pitch and applied actuation of both mirrors (the loop’s one-sample delays), the high-pass filter state of the cavity-length path, the radiation-pressure and actuation SOS delay lines;
- the Sidles–Sigg compensator state, including the running cavity-power average $\bar{P}$ (updated as $\bar{P} \leftarrow \bar{P} + (P_{\text{cav}} - \bar{P})/N_c$ with N_c ramping to 10^3) that sets its gain dynamically;
- all three controller-bank states (Section 4.3);
- all 16 reward band-filter states (Section 3.2).

A decision window is thus *not* a simulation episode — it is purely an accounting boundary at which the reward is read out and the active-arm index may change. The engine consumes the disk cache in RAM-sized chunks aligned to whole decision windows, so no window ever straddles a read boundary.

5 The fast engine

None of the above would be usable at months scale with the pure-Python per-sample loop ($\sim 2 \times 10^3$ steps/s). `fast_engine.py` ports the *exact* per-sample sequence

```
Plant.propagate → Sensors.sample_readout → SS_compensation.sample_compensation
→ Controller.sample_feedback
```

— including both one-sample delays (actuation and Sidles–Sigg compensation are applied one sample after they are computed) — into a single Numba-compiled kernel ($\sim 2\text{--}9 \times 10^5$ steps/s, i.e. up to $\sim 3500\times$ real time). Per sample it:

1. forms the beam-spot offsets from the previous pitch, the cavity length change, high-passes it, and evaluates the Fabry–Perot power response;
2. applies the radiation-pressure torque $\tau = \frac{2}{c}(P_{\text{cav}}b + d_{\text{off}}(P_{\text{cav}} - P_{\text{dc}}))$ through the rad-to-angle SOS, and the (delayed, negated) commanded actuation through the act-to-angle SOS;
3. adds the injected suspension noise (from the cache) and subtracts the delayed Sidles–Sigg compensation, yielding the new pitch;
4. projects to the eigenbasis and adds the white readout noise (Section 2.3);
5. runs the Sidles–Sigg compensator as a *unit-gain* SOS with a dynamic output scale computed from the running power average (exact in steady state);
6. steps all three controller banks and applies the active/cross-faded actuation (Section 4.3).

All filter coefficients (SOS arrays) and constants are extracted from the validated Plant-class objects at start-up, so the physics is identical *by construction*; `fast_engine.validate()` reproduces the slow engine’s steady-state scores per (regime, controller) cell. SOS filtering uses the transposed direct-form II one-sample step, matching `scipy.signal.sosfilt` state conventions exactly, so states are interchangeable between the two engines.

6 Summary of differences vs. the reference

	old_ref	Lightsaber_X
Plant physics	component graph	Plant-class refactor; validated identical
Noise	one stationary O3 batch	3 calibrated regimes, coherently synthesised, blended by a stochastic diurnal + Poisson-spike weight process; months-scale disk cache
Readout noise	spectral file	white, regime-scaled, regenerated per window from a shared seed (common random numbers across policies)
Controller	one fixed zpk	three variants (Table 2) as permanently-warm parallel banks; bumpless 2s cross-fade handover
Reward	none	16-band log-RMS score of hard-mode error + control effort, streaming stateful filters, calibrated logistic/min-max map to $[0, 1]$
Loop granularity	one batch = one run	continuous horizon; decision windows are accounting boundaries only (full state continuity)
Speed	pure Python	Numba kernel, exact port, $\sim 3500\times$ real time

Table 3: One-line summary of the modification set. The decision layer that exploits this infrastructure is documented separately.